

WHY INTERVALS?

Intervals represent ranges `[start, end]` (time slots, meeting rooms, line segments). Almost all interval problems reduce to **sorting by start or end time**, then scanning linearly in $O(N \log N)$!

THE OVERLAP CONDITION AHA

Two intervals 'A' and 'B' overlap if and only if **`max(A.start, B.start) <= min(A.end, B.end)`**!

1 3 CORE SORTING STRATEGIES

- Sort by Start Ascending** (`a[0] - b[0]`): Standard for Merge Intervals, Insert Interval, Meeting Rooms.
- Sort by End Ascending** (`a[1] - b[1]`): Optimal greedy choice for Non-Overlapping Intervals (leave max room for future!).
- Min-Heap PriorityQueue**: Track active ending times for Meeting Rooms II.

2 VISUALIZING INTERVAL MERGING

Merge [1, 3] and [2, 6]:

Interval A: [1 — 3]
Interval B: [2 — 6]
Overlap check: $2 \leq 3 \rightarrow$ OVERLAP!
Merged Result: [1 — 6] ($\max(3, 6) = 6$)

PREREQUISITES & COMPLEXITY REASONING

Operation / Algorithm	Time Complexity	Auxiliary Space
Merge Intervals (Sort + Scan)	$O(N \log N)$	$O(N)$ output
Insert Interval (Pre-sorted input)	$O(N)$ single pass	$O(N)$ output
Non-overlapping (Greedy Sort End)	$O(N \log N)$	$O(1)$ space
Meeting Rooms II (Min-Heap PQ)	$O(N \log N)$	$O(N)$ PriorityQueue

CLUES THAT SCREAM INTERVALS

- "Merge overlapping ranges" $\rightarrow$ Sort by Start + Link `end = Math.max(end, next.end)`
- "Minimum conference rooms required" $\rightarrow$ Min-Heap PQ of End Times
- "Minimum removals to eliminate overlaps" $\rightarrow$ Greedy Sort by End Time

1 INTERVAL SORTING COMPARATOR

```
// Sort by Start Time Ascending (Avoid Integer Overflow!):  
Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));  
  
// Sort by End Time Ascending:  
Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
```

Always use `Integer.compare(a, b)` instead of `a - b` to prevent integer underflow/overflow bugs!

2 MERGE INTERVALS SCANNER

```
public int[][] merge(int[][] intervals) {  
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));  
    List<int[]> merged = new ArrayList<>();  
    int[] curr = intervals[0];  
    merged.add(curr);  
    for (int[] interval : intervals) {  
        if (interval[0] <= curr[1]) { // Overlap!  
            curr[1] = Math.max(curr[1], interval[1]);  
        } else {  
            curr = interval;  
            merged.add(curr);  
        }  
    }  
    return merged.toArray(new int[merged.size()][]);  
}
```

>

PATTERN 1

INSERT INTERVAL (3-Phase Single Pass)

e.g. Insert Interval (LC 57)

3-PHASE STRATEGY

1. Add all before new interval ('interval[1] < new[0]').
2. Merge overlapping ('interval[0] <= new[1]').
3. Add all after. O(N) time!

TEMPLATE — LC 57

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] < newInterval[0]) res.add(intervals[i++]);
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.add(newInterval);
    while (i < n) res.add(intervals[i++]);
    return res.toArray(new int[res.size()][]);
}
```

PATTERN 2

MERGE INTERVALS (Sort by Start Time)

e.g. Merge Intervals (LC 56)

WHEN TO USE

Combine all overlapping intervals into single continuous intervals.

TEMPLATE — LC 56

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    int[] curr = intervals[0]; res.add(curr);
    for (int[] in : intervals) {
        if (in[0] <= curr[1]) curr[1] = Math.max(curr[1], in[1]);
        else { curr = in; res.add(curr); }
    }
    return res.toArray(new int[res.size()][]);
}
```

>

PATTERN 3

NON-OVERLAPPING INTERVALS (Greedy Sort by End Time)

e.g. Non-overlapping Intervals (LC 435), Minimum Arrows to Burst Balloons (LC 452)

WHY SORT BY END TIME

Sorting by END time leaves the MAXIMUM possible room for subsequent intervals, minimizing total removals!

TEMPLATE — LC 435

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
    int removals = 0, prevEnd = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < prevEnd) {
            removals++; // Overlap! Remove current interval (keeps smaller end)
        } else {
            prevEnd = intervals[i][1];
        }
    }
    return removals;
}
```

PATTERN 4

MINIMUM ARROWS TO BURST BALLOONS

e.g. Minimum Number of Arrows to Burst Balloons (LC 452)

ARROW INTERSECTION MECHANICS

Sort balloons by end coordinate. Shoot arrow at 'prevEnd'. If 'balloon[0] > prevEnd', need new arrow!

TEMPLATE — LC 452

```
public int findMinArrowShots(int[][] points) {
    if (points.length == 0) return 0;
    Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));
    int arrows = 1, prevEnd = points[0][1];
    for (int i = 1; i < points.length; i++) {
        if (points[i][0] > prevEnd) {
            arrows++;
            prevEnd = points[i][1];
        }
    }
    return arrows;
}
```

>

PATTERN 5

MEETING ROOMS I (Single Person Attendance Check)

e.g. Meeting Meetings (LC 252)

WHEN TO USE
Check if a single person can attend all meetings without overlap.

TEMPLATE — LC 252

```
public boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    for (int i = 0; i < intervals.length - 1; i++) {
        if (intervals[i][1] > intervals[i + 1][0]) return false;
    }
    return true;
}
```

PATTERN 6

MEETING ROOMS II (Min-Heap PriorityQueue)

e.g. Meeting Rooms II (LC 253)

MIN-HEAP STRATEGY
Sort by start time. Maintain Min-Heap storing end times of active meetings. If `start >= pq.peek()`, reuse room (`pq.poll()`)!

TEMPLATE — LC 253

```
public int minMeetingRooms(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    minHeap.offer(intervals[0][1]);
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] ≥ minHeap.peek()) {
            minHeap.poll(); // Reuse room!
        }
        minHeap.offer(intervals[i][1]);
    }
    return minHeap.size();
}
```

>

PATTERN 7

CHRONOLOGICAL LINE SWEEP (Separate Starts & Ends)

e.g. Meeting Rooms II Alternative Line Sweep

2-POINTER LINE SWEEP
Separate `starts[]` and `ends[]` arrays, sort both. Advance `startPtr`. If `starts[s] < ends[e]`, increment active rooms!

TEMPLATE — LINE SWEEP

```
public int minMeetingRoomsSweep(int[][] intervals) {
    int n = intervals.length;
    int[] starts = new int[n], ends = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = intervals[i][0]; ends[i] = intervals[i][1];
    }
    Arrays.sort(starts); Arrays.sort(ends);
    int rooms = 0, endPtr = 0;
    for (int s = 0; s < n; s++) {
        if (starts[s] < ends[endPtr]) rooms++;
        else endPtr++;
    }
    return rooms;
}
```

>

INTERVALS — WHICH PATTERN TO USE?

Start: What is the interval operation?

Combine all overlapping ranges

→

Merge Intervals

Sort by start, update `curr[1]`

+ Insert new range into sorted list

→

Insert Interval

3-phase single pass

Min removals for no overlap

→

Sort by End Time

Greedily pick earliest ending

Min meeting rooms required

→

Min-Heap PQ / Line Sweep

PQ of active end times

TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Merge overlapping"	Merge Intervals	Sort `a[0] - b[0]`, update `curr[1]`
"Insert interval"	Insert Interval	3-phase loop (before, merge, after)
"Minimum removals"	Non-overlapping	Sort `a[1] - b[1]`, track `prevEnd`
"Min meeting rooms"	Meeting Rooms II	Min-Heap storing ending times

INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why sort by end vs start for non-overlapping?	Sorting by END time leaves the maximum remaining time available for future non-overlapping intervals!
PQ vs Line Sweep for Meeting Rooms II?	Min-Heap PQ uses $O(N \log N)$ time and $O(N)$ space. Line Sweep separates arrays and is often faster in practice!

>

EASY & MEDIUM — Meeting Rooms & Merge

LC 252 · Meeting Rooms

EASY

Pattern: Single Person Check
Key Insight: Sort start. 'curr.end > next.start' = false.

```
public boolean canAttendMeetings(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    for (int i = 0; i < intervals.length - 1; i++) {
        if (intervals[i][1] > intervals[i + 1][0]) return false;
    }
    return true;
}
```

LC 56 · Merge Intervals

MEDIUM

Pattern: Merge Overlapping
Key Insight: Update 'curr[1] = Math.max(...)'.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    int[] curr = intervals[0]; res.add(curr);
    for (int[] in : intervals) {
        if (in[0] <= curr[1]) curr[1] = Math.max(curr[1], in[1]);
        else { curr = in; res.add(curr); }
    }
    return res.toArray(new int[res.size()][]);
}
```

LC 57 · Insert Interval

MEDIUM

Pattern: 3-Phase Single Pass
Key Insight: Before, overlap merge, after.

```
public int[][] insert(int[][] in, int[] newIn) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = in.length;
    while (i < n && in[i][1] < newIn[0]) res.add(in[i++]);
    while (i < n && in[i][0] <= newIn[1]) {
        newIn[0] = Math.min(newIn[0], in[i][0]);
        newIn[1] = Math.max(newIn[1], in[i][1]);
        i++;
    }
    res.add(newIn);
    while (i < n) res.add(in[i++]);
    return res.toArray(new int[res.size()][]);
}
```

>

MEDIUM — Minimum Arrows to Burst Balloons

LC 452 · Minimum Arrows for Balloons

MEDIUM

Pattern: Sort by End

```
public int findMinArrowShots(int
```

MEDIUM — Meeting Rooms II & Non-Overlapping

LC 253 · Meeting Rooms II

MEDIUM

Pattern: Min-Heap PQ
Key Insight: If 'start >= pq.peek()', reuse room ('pq.poll()').

```
public int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    pq.offer(intervals[0][1]);
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= pq.peek()) pq.poll();
        pq.offer(intervals[i][1]);
    }
    return pq.size();
}
```

LC 435 · Non-overlapping Intervals

MEDIUM

Pattern: Greedy Sort by End
Key Insight: If overlap, 'removals++'.

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
    int rem = 0, prevEnd = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < prevEnd) rem++;
        else prevEnd = intervals[i][1];
    }
    return rem;
}
```

>

🔍 DRY RUN — Merge Intervals ([[1,3],[2,6],[8,10]])

Step	Interval	curr Interval	Overlap Check	Result List State
1	[1, 3]	[1, 3]	Initial	'[[1, 3]]'
2	[2, 6]	[1, 3]	$2 \leq 3 \rightarrow$ Overlap! 'curr[1] = max(3, 6) = 6'	'[[1, 6]]'
3	[8, 10]	[1, 6]	$8 > 6 \rightarrow$ No overlap! 'curr = [8, 10]'	'[[1, 6], [8, 10]]' ✅

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Use 'Integer.compare(a, b)' for sorting

☐ Write Merge Intervals (LC 56) in 3m

☐ Code Insert Interval 3-phase loop

☐ Write Meeting Rooms II with Min-Heap PQ

☐ Code Non-overlapping Intervals (sort by end)

☐ Implement Minimum Arrows to Burst Balloons

☐ Code Meeting Rooms II Line Sweep alternative

☐ Prove why sorting by end time is greedy optimal

📐 MATHEMATICAL PROOF — GREEDY END SORT FOR NON-OVERLAPPING

Claim: Sorting by end time E_i maximizes the number of non-overlapping intervals.

Proof: Choosing the interval with the EARLIEST end time E_1 leaves the maximum possible timeline available for all remaining intervals. Any alternative choice ending later can only restrict future choices $\rightarrow$ **Strictly Optimal Greedy Choice!**

📦 GOLDEN RULES — NEVER FORGET

Rule 1 — Integer.compare Prevents Overflow
Never use '(a, b) -> a[0] - b[0]!' If '\$a[0] = -2147483648\$', subtraction overflows. Always use 'Integer.compare(a[0], b[0])!'

Rule 2 — Strict vs Non-Strict Overlap
Meeting Rooms II considers '[1, 5]' and '[5, 10]' NON-overlapping (room reused at 5). Balloon shooting considers '[1, 5]' and '[5, 10]' OVERLAPPING (arrow at 5 bursts both)!